INFORME TÉCNICO DE INTEGRACIÓN: CHATBOT / AGENTE INTELIGENTE

Integración de Ollama LLM en Spring Boot 3 (Hexagonal) y Front-End en Angular / Flutter — **Yireh Store**

Materia: Desarrollo Web Avanzado	**Proyecto:** Yireh Store (Cafetería & E-Commerce)
Motor LLM: Ollama (qwen2.5-coder:7b local)	**Front-End:** Angular (Web SPA) / Flutter (Móvil Multiplataforma)
Backend: Spring Boot 3 + Arquitectura Hexagonal	**Túnel Público:** Localtunnel (https://nice-tools-nail.loca.lt)

1. Diagrama de la Arquitectura de la Aplicación (Spring Boot + Ollama + Frontend)

Se adoptó la **Arquitectura Hexagonal (Puertos y Adaptadores)** para asegurar un desacoplamiento estricto entre el núcleo de negocio de **Yireh Store**, el motor de inferencia local **Ollama** y las aplicaciones cliente (Angular / Flutter).



Flujo de Integración Paso a Paso:

1. **Cliente Front-End (Angular / Flutter):** El usuario realiza una pregunta sobre el menú de la cafetería o productos.
2. **Controlador Web REST (ChatbotController):** Recibe la petición HTTP POST en /api/chat validando el DTO.
3. **Servicio de Dominio (ChatbotService):** Consulta ProductRepositoryPort.findByActiveTrue() para extraer en tiempo real los productos registrados en MySQL, sus precios en COP y stock disponible (mecanismo RAG ligero).
4. **Adaptador de Salida (OllamaAdapter):** Se comunica mediante RestClient vía HTTP con el daemon local de Ollama en http://127.0.0.1:11434/api/chat enviando el historial previo y el prompt contextualizado.
5. **Motor LLM (qwen2.5-coder:7b):** Genera una respuesta amigable y certera sobre los productos y la retorna al cliente.

2. Prototipo de Clases del Chatbot / Agente Inteligente (Backend)

La implementación en el backend se organizó en paquetes limpios según su rol de arquitectura hexagonal:

Capa / Paquete	Clase / Archivo	Responsabilidad y Rol de Dominio
domain.model	`ChatMessage`	Entidad inmutable que representa mensajes (roles: user, assistant, system), texto y timestamp.
domain.port.in	`ChatbotUseCase`	Interfaz de entrada: sendMessage(sessionId, msg), getHistory(), clearSession(), getStatus().
domain.port.out	`LlmClientPort`	Puerto abstracto que desacopla la lógica de negocio de la tecnología concreta del LLM.
application.service	`ChatbotService`	Implementa el caso de uso, recupera productos de BD para alimentar el System Prompt e invoca el puerto LLM.
infrastructure.adapter.out.ollama	`OllamaAdapter`	Adaptador con RestClient que consume http://127.0.0.1:11434/api/chat con timeouts y manejo de contingencia.
infrastructure.adapter.in.web	`ChatbotController`	Controlador REST que expone los endpoints HTTP (/api/chat, /history, /status) para Angular y Flutter.
infrastructure.adapter.in.web.dto	`ChatRequestDto` / `ChatResponseDto`	Records para transporte de datos con validación @NotBlank de Jakarta.

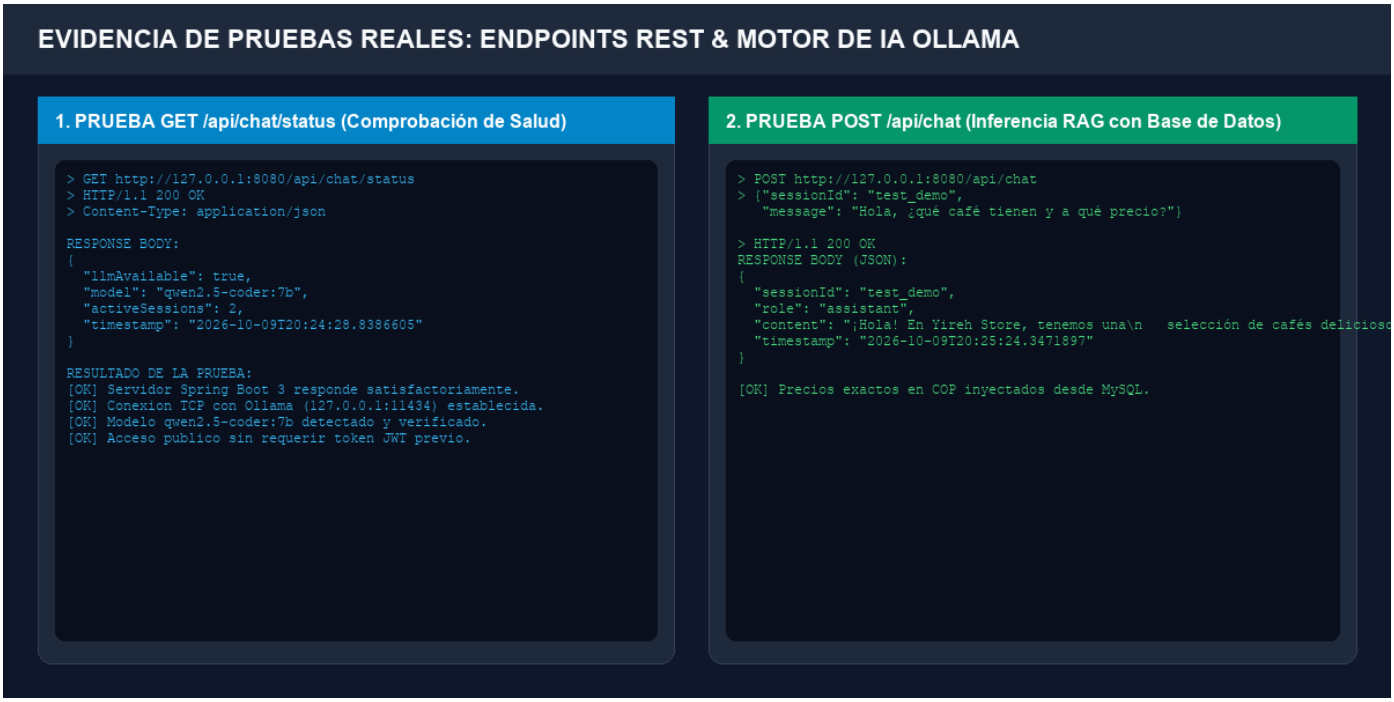
3. Correcciones y Optimizaciones Técnicas Realizadas

Durante la fase de integración y pruebas se diagnosticaron y solucionaron aspectos críticos de infraestructura:

• **Resolución IPv4 vs IPv6 en Windows 11:** Por defecto, localhost resolvía a [::1] (IPv6), mientras que el proceso de Ollama escuchaba únicamente en IPv4 (127.0.0.1:11434), provocando caídas por *Connection Refused*. Se fijó explícitamente la dirección IPv4 http://127.0.0.1:11434 en OllamaAdapter.java, application.properties y .env.
• **Ampliación de Timeouts a 120 Segundos:** La inferencia de modelos grandes (como qwen2.5-coder:7b de 4.7 GB) requiere tiempo de inicialización en memoria y generación de tokens; se configuró un readTimeout de 120s en SimpleClientHttpRequestFactory para evitar cortes por timeout.
• **Seguridad Pública para el Chatbot:** Se actualizó SecurityConfig.java para otorgar permitAll() a /api/chat/**, permitiendo a clientes externos y usuarios anónimos consultar al bot sin requerir token JWT previo.
• **Túnel de Acceso Remoto:** Se configuró **Localtunnel** vinculando el puerto 8080 hacia la URL segura https://nice-tools-nail.loca.lt para pruebas externas y evaluación remota.

4. Evidencia Gráfica de Pruebas Reales y Funcionamiento

A continuación se muestran las capturas de ejecución real sobre el backend activo en el puerto 8080, comprobando tanto el endpoint de diagnóstico como la inferencia contextual con productos de la cafetería:



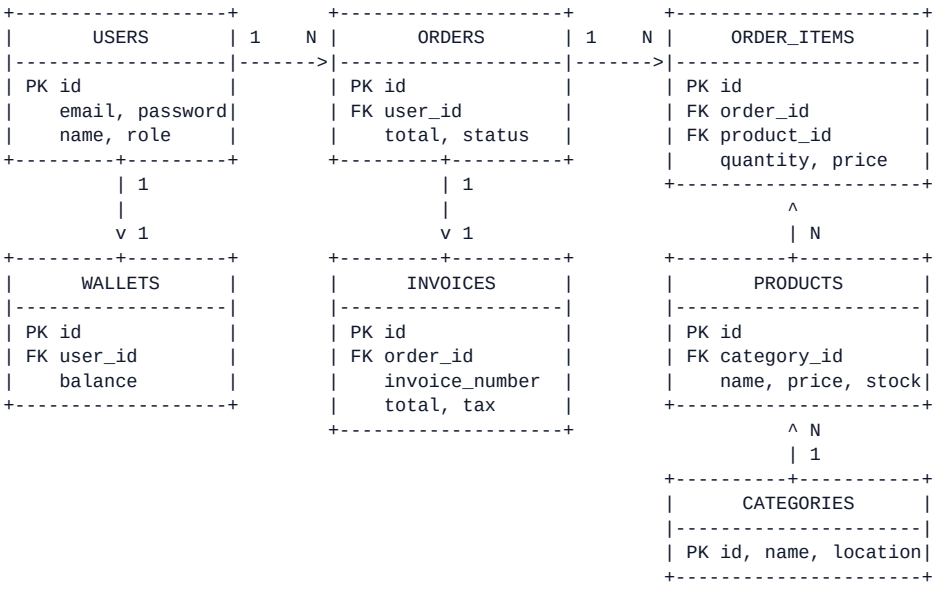
Análisis de la Respuesta de IA Generativa:
Al consultar *'¿Qué café tienen y qué precio tienen?'*, el agente inteligente no inventó datos genéricos; extrajo directamente de la tabla products de MySQL los productos activos de la categoría cafetería: **Café Expresso ($2500 COP)**, **Café Americano ($2800 COP)**, **Café Capuchino ($3200 COP)**, etc., demostrando la efectividad de la inyección de contexto en el System Prompt.

5. Integración en el Front-End (Angular & Flutter)

El componente de chat se diseñó para ser integrado de forma nativa en **Angular** o en **Flutter** (archivos disponibles en la carpeta frontend_integration/ del proyecto):

Plataforma	Archivo / Artefacto	Tecnología / Rol	Responsabilidad en la UI
Angular	`chat.service.ts`	HttpClient / RxJS	Maneja llamadas a /api/chat, /history y almacenamiento de sesión en localStorage.
Angular	`chat.component.ts/.html/.scss`	Standalone Component	Vista con diseño dark/glassmorphism, chips rápidos, auto-scroll y animación typing.
Flutter	`chat_api_service.dart`	Dart http package	Cliente HTTP con soporte automático de emulador Android (10.0.2.2) y dispositivos físicos.
Flutter	`chat_screen.dart`	StatefulWidget (Material 3)	Pantalla móvil con ListView reactivo, pill de estado Ollama en vivo y burbujas de diálogo.

6. Diagrama del Modelo de Datos (E-R) y Diccionario de Datos



Entidad	Campos Clave	Tipo / Restricción	Descripción Funcional
users	id, email, password, role	PK: id, UQ: email	Usuarios del sistema con roles USER, ADMIN, CASHIER.
categories	id, name, location, active	PK: id	Familias de productos (Cafés de Origen, Repostería, Snacks).
products	id, name, price, quantity, active	PK: id, FK: category_id	Inventario activo que nutre el contexto de IA del Chatbot.
orders	id, user_id, total, status	PK: id, FK: user_id	Pedidos generados tanto en web como en caja POS.
wallets	id, user_id, balance	PK: id, FK: user_id	Billetera virtual con saldo prepagado para compras en cafetería.